aTableau

Andrew Mathas

Version 2.0-alpha

This version of the package is already quite powerful, but it is still in the proof-of-concept phase. There are bugs and there is very little documentation. Several planned features have not been implemented and some things that have been implemented are very rough. Everything is subject to change.

The package will eventually be released on ctan. Comments and suggestions are welcome.

This package provides commands for drawing Young diagrams, tableaux, abacuses and other common objects used in algebraic combinatorics and representation theory. The commands are designed to be easy to use and easy to read. The literature contains many embellishments of the basic diagrams, so we provide mechanisms to customise and style these diagrams.

The package provides basic commands for drawing these objects, together with styling options and a key-value interface for adding extra features. Under the hood, everything is drawn using TikZ. It is possible to use all of our commands either as standalone commands, or as components of a `tikzpicture` environment. This makes it possible to use these commands inside more complicated diagrams.

Contents

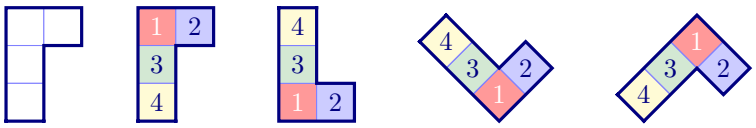
1. Young diagrams and tableaux

Partitions, which are weakly decreasing sequences of non-negative integers, are fundamental objects in algebraic combinatorics and representation theory. Rather than working with sequences of integers, it is often useful to visualise partitions as Young *diagrams*, which are arrays of boxes in the plane. A *tableau* is a (Young) diagram where the boxes are labelled using some alphabet. Equivalently, a diagram is a tableau with empty labels.

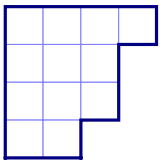
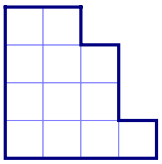
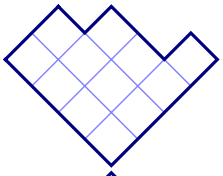
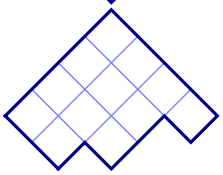
For example, $\lambda = (4, 3, 3, 2, 0, 0, \dots)$ is a partition of 12, since the entries sum to 12. It is customary to omit the zeros when writing partitions and to use exponents for repeated parts, so we also write $\lambda = (4, 3^2, 2)$. Many authors identity a partition $\lambda = (\lambda_1, \lambda_2, \dots)$ with its *diagram*, which is the set

$$[\lambda] = \{\, (r, c) \,|\, 1 \le c \le \lambda_r \text{ for } r \ge 1 \,\}.$$

The diagram of λ is visualised as a justified array of boxes in the plane, where the first row has 4 boxes, the nest two rows have 3 boxes and the last row has 2 boxes. The *shape* of a diagram is the partition that gives the number of boxes in each row. Here are some examples of the diagrams and tableaux of shape $(2, 1^2)$ that can be drawn using aTableau:



This choice of colouring is gratuitous, but it emphasises that aTableau allows you to add style (and, in particular, colour), to every box in these diagrams. This example also shows that aTableau supports the four different conventions used for drawing Young diagrams and tableaux. The following table shows how to draw the Young diagram of shape $\lambda = (4, 3^2, 2)$ using the four tableaux conventions supported by aTableau:

Convention	aTableau command	Diagram
English	`\Diagram[english]{4,3^2,2}`	
French	`\Diagram[french]{4,3,3,2}`	
Ukrainian[1]	`\Diagram[ukrainian]{4,3^2,2}`	
Australian[2]	`\Diagram[australian]{4,3,3,2}`	

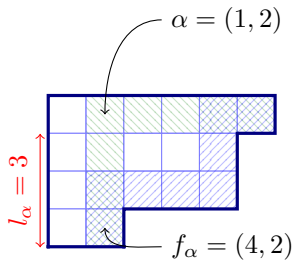
As shown, partitions can either be typed out in full or using the, typically shorter, exponential notation. The different conventions for diagrams, and tableaux, are listed in order of their popularity in the literature. By default, the *English* convention is used, so `\Diagram{4,3,3,2}` produces the first diagram above. These four conventions can be applied in the same way to the `\Tableau` command. You can change the default convention using the `\aTabSet` command.

This table shows how the commands provided by aTableau work: there are basic commands for drawing diagrams, and these diagrams can be embellished using optional arguments and a key-value interface. In addition, as the following example shows, by supplying (optional) Cartesian coordinates, these diagrams can be used inside a `tikzpicture` environment to create more exotic pictures:

```
% \usetikzlibrary{patterns}
\begin{tikzpicture}[
    B/.style={pattern=north east lines, pattern color=blue,opacity=0.5},
    Y/.style={pattern=north west lines, pattern color=ForestGreen,opacity=0.5},
  ]
  \Diagram(0,0)[ribbons={Y|16ccccrrr, B|16crrcccr}]{6,5^2,2}
  \draw[->](1.5, 1)node[right]{  $\alpha=(1,2)$} to [out=180,in=90]  (B-1-2.base);
  \draw[->](1.5,-2)node[right]{$f_\alpha=(4,2)$} to [out=180,in=270] (B-4-2.base);
  \draw[red,<->]([xshift=-1mm]B-1-1.south west)
     --node[rotate=90,above]{$l_\alpha=3$}([xshift=-1mm]B-4-1.south west);
\end{tikzpicture}
```

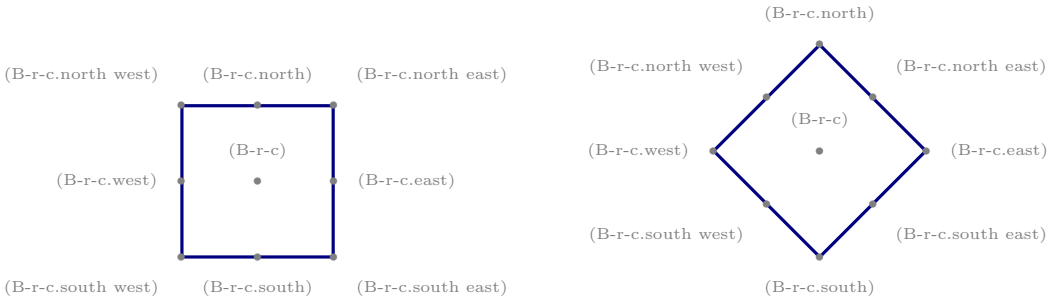
[1] Some authors call this the Russian convention

[2] I have not seen this convention in the literature, but this name for the convention seems apt.



Most of the commands in this example are TikZ commands, but most of the picture is drawn by the `\Diagram` command. The top of the example defines two TikZ styles, B and Y, which are used in the ribbons. The Young diagram is placed at position $(0, 0)$ by the `\Diagram` command. The three `\draw` commands in this example are used to add the three arrows to the picture. You should consult the very readable TikZ manual for an explanation of how these commands work.

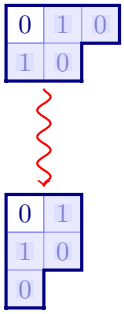
From the viewpoint of aTableau, the most important point of this example is that the `\Diagram` command can be used inside a `tikzpicture` environment because it is equipped with the Cartesian coordinate $(0, 0)$. The example also uses the four named coordinates, (B-1-1), (B-1-2), (B-4-1), and (B-4-2). When aTableau draws a tableau, or a diagram, it creates TikZ named coordinates for each of the boxes in the diagram, so that (B-r-c) refers to the box in row r and column c. Giving finer control, the following standard TikZ *anchors* are available:



Here, r and c should be replaced by the row and column index of a box in the tableau. These anchors are only defined for the nodes in the tableau. These *anchors* are a standard part of TikZ. If you change the shape of the node then the available anchors may change; for more information see the TikZ manual.

The previous example also shows that the `\Diagram` command accepts a ribbons option, which adds *ribbons* to the diagram. Moreover, these ribbons can be given TikZ styling. All of the options to these commands are described in the following sections.

In the example above the Cartesian coordinate $(0, 0)$ is necessary because the `\Diagram` command is embedded inside a `tikzpicture` environment, but the coordinate itself is not important in the sense that changing the coordinate will not change the picture. Cartesian coordinates become more meaningful when there are more components inside the `tikzpicture` environment. Consider:

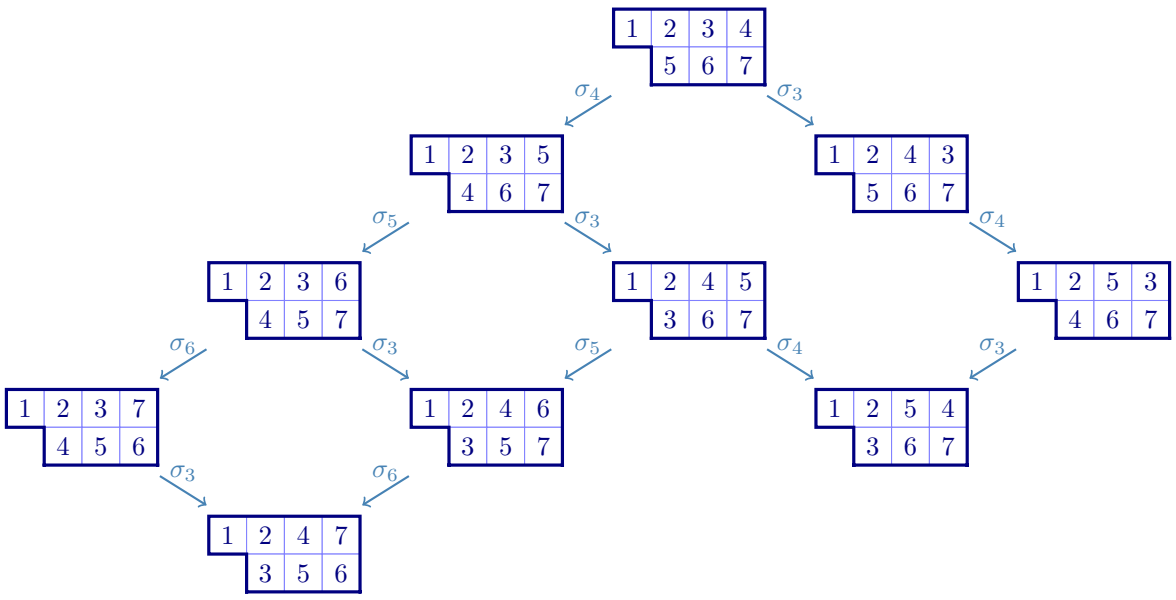



```
% \usetikzlibrary{decorations.pathmorphing}
\begin{tikzpicture}[wave/.style={thick,red,->,decorate,decoration=snake}]
  \aTabSet{ribbonstyle={fill=blue!20, opacity=0.4}}
  \Diagram(0,0)[e=2,show=residues,ribbons={13crc},prefix=A]{3,2}
  \Diagram(0,-2.5)[e=2,show=residues,ribbons={12rcr},prefix=B]{2^2,1}
  \draw[wave](0.5,-1.1)--(0.5, -2.4);
\end{tikzpicture}
```

The following sections describe the commands defined by the package and how to use them. We start with the `\Tableau` command because all of the diagram and tableau commands, except for `\RibbonTableau`, are derived from the `\Tableaux` command.

Finally, the condition that coordinates are required only when these diagrams are being used inside a `tikzpicture` environment needs to be interpreted appropriately. For example, coordinates are not required when the diagrams are inside nodes in these diagrams:

```
% \usetikzlibrary{matrix}
\begin{tikzpicture}[arr/.style={->,SteelBlue, thick}]
  \matrix (M)[matrix of nodes,row sep=4mm,column sep=4mm]{
    &    &    &\ShiftedTableau{1234,567} \\
    &    & \ShiftedTableau{1235,467}
    &    & \ShiftedTableau{1243,567}\\
    &  \ShiftedTableau{1236,457}
    &    & \ShiftedTableau{1245,367}
    &    & \ShiftedTableau{1253,467}\\
    \ShiftedTableau{1237,456}
    &    & \ShiftedTableau{1246,357}
    &    &\ShiftedTableau{1254,367}\\
    &\ShiftedTableau{1247,356}\\
   };
   \draw[arr](M-1-4)--node[above]{$\sigma_4$}(M-2-3);
   \draw[arr](M-1-4)--node[above]{$\sigma_3$}(M-2-5);
   \draw[arr](M-2-3)--node[above]{$\sigma_5$}(M-3-2);
   \draw[arr](M-2-3)--node[above]{$\sigma_3$}(M-3-4);
   \draw[arr](M-2-5)--node[above]{$\sigma_4$}(M-3-6);
   \draw[arr](M-3-2)--node[above]{$\sigma_6$}(M-4-1);
   \draw[arr](M-3-2)--node[above]{$\sigma_3$}(M-4-3);
   \draw[arr](M-3-4)--node[above]{$\sigma_5$}(M-4-3);
   \draw[arr](M-3-4)--node[above]{$\sigma_4$}(M-4-5);
   \draw[arr](M-3-6)--node[above]{$\sigma_3$}(M-4-5);
   \draw[arr](M-4-1)--node[above]{$\sigma_3$}(M-5-2);
   \draw[arr](M-4-3)--node[above]{$\sigma_6$}(M-5-2);
\end{tikzpicture}
```



1a. **Tableaux.** This section describes the `\Tableau` command, which draws tableau or labelled diagrams. The syntax for this command is:

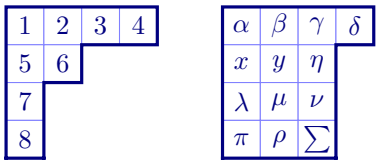
`\Tableau (x,y) [options] {tableau entries}`

(x, y) :
The Cartesian coordinates (x, y) are needed if and only if the tableau is inside a `tikzpicture` environment.

options :
Optional arguments, described below.

tableau entries :
The `tableau entries` are given as a comma separated list, with commas separating rows and each *token*, or braced-group of tokens, being in a separate column. As we explain below, optionally, each tableau entry can be prefixed by TikZ styling commands.

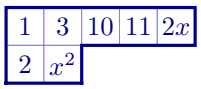
We show by example how to use the `\Tableau` command, starting with basic usage and finishing with style. Inside `\Tableau`, the rows are separated by commas, with the column entries given by the "letters" in the entry for each row:




```
\Tableau{ 1234, 56, 7, 8 }
\Tableau{ \alpha\beta\gamma\delta,
          xy\eta,
          \lambda\mu\nu,
          \pi\rho\sum }
```

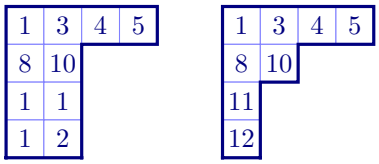
You can put all of the tableau specifications on one line, without any spaces. Alternatively, as I do, you add spaces and line breaks between the tableau entries to improve readability (although, you cannot have blank lines inside a `\Tableau` command). As this example suggests, by default, the entries of a tableau are typeset as mathematics, but this can be changed.

As the second example above shows, the tableau entries are *not* just individual characters: they also include TEX commands. Tableaux frequently contain entries that are not single characters, or given by commands. Such entries can be inserted in a tableau by enclosing them in braces:



```
\Tableau{ 13{10}{11}{2x}, 2{x^2} }
```

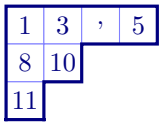
There is one additional caveat for braced-entries: *any single braced entry in a row needs to be double-braced.* In practical terms, the need to double-bracing is shown by the following example:




```
\Tableau{ 1345, 8{10}, {11}, {12} }
\Tableau{ 1345, 8{{10}}, {{11}}, {{12}} }
```

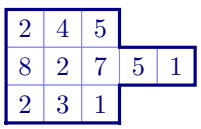
The {11} and {12} in the first tableau are both treated as spanning two columns in the tableau because they are not doubled-braced; in contrast,they occupy only one column in the second tableau because they are double-braced.

The need for double-bracing of singleton entries is a consequence of how LATEX3 detects braced commas in comma separated lists. If you want to have a comma as an entry in a tableau, then you should enclose it in braces.



```
\Tableau{ 13{,}5, 8{10}, {{11}} }
```

As we have defined them, tableaux are always of *partition* shape, in that the lengths of the rows form a weakly decreasing sequence. In fact, the `\Tableau` command also accepts tableaux of *composition* shape, where the lengths of the rows are not necessarily in decreasing order.

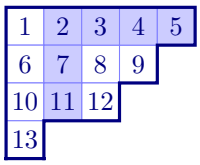


```
\Tableau{ 245,82751,231 }
```

Empty rows in tableaux and Young diagrams are not supported!

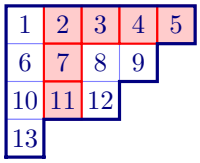
Adding style. The aTableau package provides several different ways to add TikZ style commands to tableaux and diagrams. This is made possible by adding style prefixes to the tableau entries. To take advantage of these styles you will need at least rudimentary knowledge of how TikZ styles work. At least to get started, the examples in this manual will help.

The simplest way to change the style of a tableau entry is to give it a ∗-prefix in the tableau specifications. This is the easiest way to add some style to some of the nodes in the tableau:




```
\Tableau{ 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

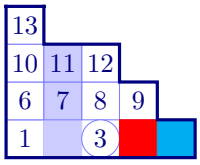
By default, the ∗-entries are given the TikZ styling `fill=blue!20`. You can change the default ∗-style using the starstyle key:




```
\Tableau[starstyle={fill=red!20,draw=red,thick}]
        { 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```

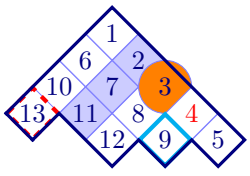
Notice that the ∗-styling takes precedence over the default styling of the surrounding boxes. This is because the boxes with the default styling are placed first, in the order that they are entered, after which the boxes with styling are placed.

The ∗-syntax is convenient if you want to emphasise some of the boxes in a tableau, using a single style. In fact, it is possible to give every box in the tableau its own style by putting the style specification inside square brackets, `[...]`, before the corresponding letter in the tableau specification. You can mix the ∗-syntax and the [....]-style syntax for different entries, although you can only apply one of these style settings to any given node.




```
\Tableau[french]{
    1[blue!20]2[circle]3[red]4[cyan]5,
    6*789,
    {10}*{11}{12},
    {{13}} }
```

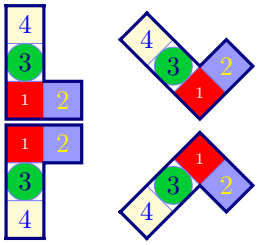
This example shows that some precision is needed when changing the style because simply giving a colour will change both the colour used to fill the box and the text colour, which is why some of tableau entries above appear to be blank. Instead of using just a colour, use `text=...` to change the text colour, `fill=...` to change the fill colour, and `draw=...` to change the border colour. If you want to apply more than one style setting to any tableau entry, such as `circle,fill=orange`, then you need to enclose the all of style settings inside matching square and curly braces `[\{...\}]`. This is necessary because commas are used to separate the rows of the tableau.




```
\Tableau[australian]{
    1[fill=blue!20]2[{circle,fill=orange}]3[text=red]45,
    6*78[{draw=cyan,ultra thick}]9,
    {10}*{11}{12},
    [{dashed,draw=red,ultra thick}]{13} }
```

As this example shows, modifying the borders does not add very much emphasis because not very prominent. The orange circle looks too big, but it is what we asked for because it diameter matches the central width of the box. The next example shows that we get a better result for *Australian* and *Ukrainian* tableaux if we add a *minimum size* specification.

For complicated styles, or styles that are used often, we recommend defining the style outside of the `\Tableau` command by using the `\tikzset` command to define the style:



```
\tikzset{
   B/.style={fill=blue!40, text=yellow},
   G/.style={fill=green!80!blue,circle,minimum
size=5mm},
   R/.style={fill=red,text=white, font=\tiny},
   Y/.style={fill=yellow!20, text=blue}
}
\Tableau[french]{[R]1[B]2,[G]3,[Y]4}
\Tableau[ukrainian]{[R]1[B]2,[G]3,[Y]4}
\Tableau[english]{[R]1[B]2,[G]3,[Y]4}
\Tableau[australian]{[R]1[B]2,[G]3,[Y]4}
```

In this example, the TikZ styles B, G R, and Y are being applied to the tableaux entries labelled 1, 2, 3, and 4, respectively. The colours used here, and throughout this manual, are for demonstration purposes only: I recommend against such garish choices in any self-respecting document!

Tableau options. The optional ∗-styles and [...]-styles are the main mechanism for styling individual nodes in a tableau. Using a key-value syntax, you can change other aspects of the tableaux produced by the `\Tableau` command. The rest of this section describes these keys, their default values, and gives examples of how to use them.

The options described below can be used with the `\Tableau` command by placing them as a comma separated list inside square brackets:

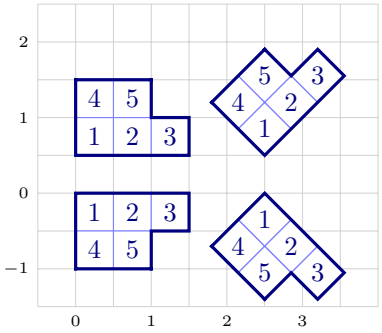
`\Tableau[options]{tableau entries}]` or `\Tableau(x,y)[options]{tableau entries}]`.

The options can appear in any order, with later options having precedence over earlier ones.

When used inside a `\Tableau` command, the options only affect that particular tableau. Most of these options can also be used in the `\aTabSet` command, to change the default settings of all following tableaux in the same LaTeX group. For example, if you put the command `\aTabSet[ukrainian]` in the preamble of your document then, by default, latter tableaux and Young diagrams will be drawn using the Ukrainian convention.

(x, y)

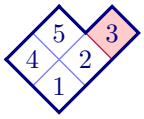
The (x, y)-coordinates are needed if, and only if, `\Tableau` command is used inside a `tikzpicture` environment, in which case (x, y) gives the coordinates of the "outside corner" of the box in row 1 and column 1 of the tableau. The following example shows how the tableaux are placed using coordinates inside a `tikzpicture` environment. The example also shows that, by default, the tableau boxes are half a unit wide.



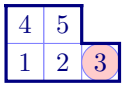
```
\begin{tikzpicture}[addgrid]
  \Tableau(0,0)     [english]   {123,45}
  \Tableau(0,0.5)   [french]    {123,45}
  \Tableau(2.5,0.5)[ukrainian] {123,45}
  \Tableau(2.5,0)[australian]{123,45}
\end{tikzpicture}
```

english (default: english)
french
ukrainian
australian

These four options change the convention that is used to draw the tableaux. By default, the english convention is used. This can be changed using `\aTabSet`. The capitalised names, `Australian`, `English`, `French` and `Ukrainian`, are also recognised.

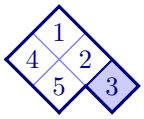



```
\tikzset{R/.style={fill=red!20,draw=red}}
\Tableau[ukrainian]{12[R]3,45}
```




```
\tikzset{R/.style={fill=red!20,circle, radius=0.2}}
\Tableau[french]{12[R]3,45}
```

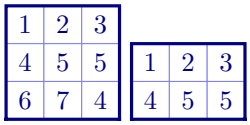

```
\tikzset{B/.style={fill=blue!20,draw=NavyBlue, thick}}
\Tableau[english]{12[{B, thick}]3,45}
```



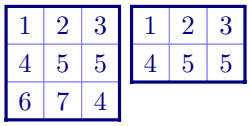

```
\tikzset{B/.style={fill=blue!20,draw=NavyBlue, thick}}
\Tableau[australian]{12[{B, thick}]3,45}
```

align (default: center) [accepts: centre/bottom/top]

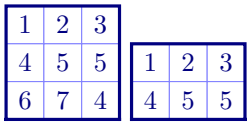
By default, tableaux are centred vertically. This can be changed using the align key.



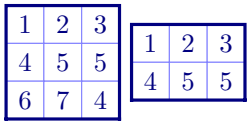

```
\Tableau{123,455,674}
\Tableau{123,455}
```




```
\Tableau[align=top]{123,455,674}
\Tableau[align=top]{123,455}
```




```
\Tableau[align=bottom]{123,455,674}
\Tableau[align=bottom]{123,455}
```

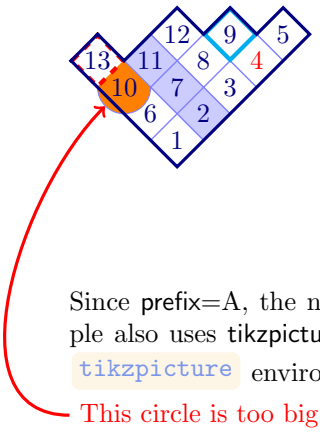



```
\Tableau[align=centre]{123,455,674}
\Tableau[align=centre]{123,455}
```

It is not necessary to have the same align keys on each tableau, but this one way to ensure that you get the desired result. Here and elsewhere, American spelling variations are also accepted.

prefix (default: B) [accepts: string/character]

By default, the boxes can be referenced using the node names (B-1-1), (B-1-2), (B-1-3), ..., with (B-*r*-*c*) referring to the node in row *r* and column *c*. This feature is only useful when the tableau is inside a `tikzpicture` environment, or if the TikZ *remember picture* style is being used, which the next example does.




```
\Tableau[ukrainian, prefix=A,
  tikzpicture={remember picture}]
  { 1[fill=blue!20]23[text=red]45,
    6*78[{draw=cyan,ultra thick}]9,
    [{circle,fill=orange}]{10}*{11}{12},
    [{dashed,draw=red,ultra thick}]{13}
  }
```

Since prefix=A, the nodes in this tableau are referenced as (A-1-1), (A-1-2), (A-1-3), ... As the example also uses tikzpicture=*remember picture*, we can reference the node names in last tableau from other `tikzpicture` environments. For example, we can do things like this:

```
\tikz[remember picture, overlay]
  \draw[very thick, ->,red]
       (0,0)node[right]{This circle is too big!}
       to [out=180, in=225](A-3-1);
```

tikzpicture [accepts: TikZ style commands]

Use this command to add an optional argument to the underlying `tikzpicture` environment that contains the tableau.

tikz before [accepts: TikZ commands]
tikz after [accepts: TikZ commands]

inner wall

outer wall

inner style

outer style

fill

text

font

math nodes (default: true) [accepts: true/false]

text nodes [accepts: true/false]

shape

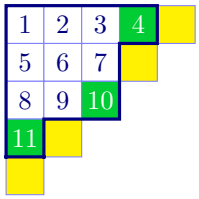
node height

node width

ribbons [accepts: a comma separated list of ribbon specifiers]
snobs [accepts: a comma separated list of ribbon specifiers]
ribbonstyle (default: fill=blue!20) [accepts: TikZ style commands]

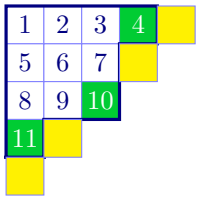
Use the ribbons key to add a comma separated list of ribbons to your tableau. The syntax for adding ribbons is described in subsection 1f, which describes the `\RibbonTableau` command. The basic idea is that you specify the location of the head of the ribbon, which has minimum row index and maximum

column index, and then gives a sequence of r's and c's depending on whether the ribbon moves along a row or column – and, of course, you can add style. Use the ribbonstyle option to change the default style of the ribbons. For example, noting that a box is a ribbon of length 1, the following code uses ribbons of length one (that is, nodes), to highlight the addable nodes of this tableau.




```
\Tableau[starstyle={fill=green!80!blue,text=white},
         ribbonstyle={fill=yellow},
         ribbons={15,24,42,51},
        ]
        {123*4,567,89*{10},*{11}}
```

Use snobs to add ribbons to the tableau that are placed *after* the outline of the diagram has been drawn. (Writing *ribbons* backwards gives *snob*bir.) The default style for a `snobs` ribbon is given by `ribbonstyle`.



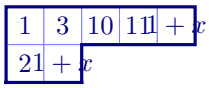

```
\Tableau[starstyle={fill=green!80!blue,text=white},
         ribbonstyle={fill=yellow},
         snobs={15,24,42,51},
        ]
        {123*4,567,89*{10},*{11}}
```

The main difference between these two pictures is that the tableau boundary is that when you use snobs, which is because the styling of these boxes overwrites the boundary.

star style

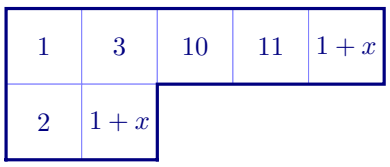
scale (default: 1) [accepts: a decimal number]
xscale (default: 1) [accepts: a decimal number]
yscale (default: 1) [accepts: a decimal number]

By design, boxes in tableaux do not automatically resize to fit their contents. For example, consider:

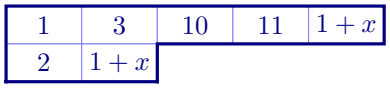



```
\Tableau{ 13{10}{11}{1+x}, 2{1+x} }
```

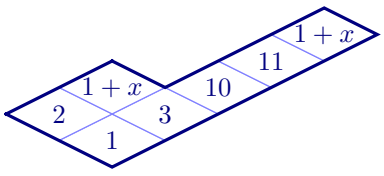
This output is clearly not desired. The options scale, xscale and yscale can be used to rescale the boxes in tableaux and diagrams. The names are slightly misleading because xscale rescales in the direction of increasing column index and yscale rescales in the direction of increasing row index.




```
\Tableau[scale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

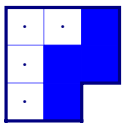



```
\Tableau[xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

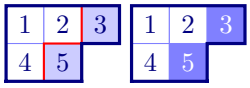



```
\Tableau[ukrainian, xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```

The options, scale, xscale, and yscale, affect all aTableau diagrams. If you want to use `\aTabSet` to chbnge the sizes of all of the diagrams and tableaux in your document it may be better to use the box height and box width options.

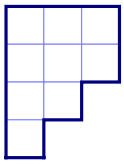


```
\Tableau[starstyle={blue}]{..*., .*.*., .*.}
```

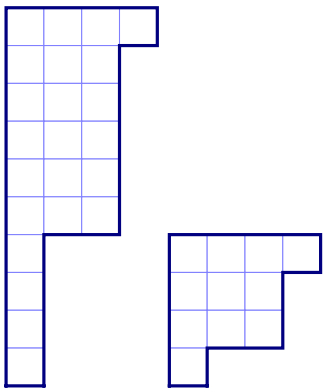



```
\tikzset{R/.style={fill=blue!20,draw=red,thick}}
\Tableau{12[R]3,4[R]5}
\Tableau[starstyle={fill=blue!50,text=white}]{12*3,4*5}
```

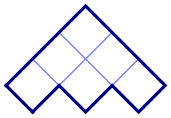
1b. **Young diagrams.**



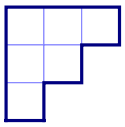
```
\Tableau{~~~,~~~,~~,~}
```



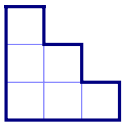
```
\Diagram{4,3^5,1^4}
\Diagram{4,3,3,1}
```



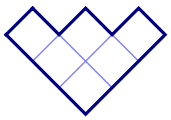
```
\Diagram[australian]{3,2,1}
```



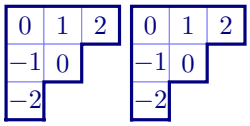
```
\Diagram[english]{3,2,1}
```



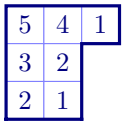
```
\Diagram[french]{3,2,1}
```



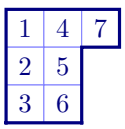
```
\Diagram[ukrainian]{3,2,1}
```



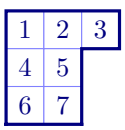

```
\Diagram[show=contents]{3,2,1}
\Tableau{012,{-1}0,{{-2}}}
```



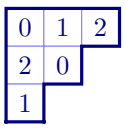

```
\Diagram[show=hooks]{3,2,2}
```




```
\Diagram[show=final]{3,2,2}
```

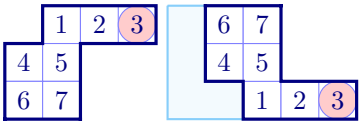



```
\Diagram[show=initial]{3,2,2}
```



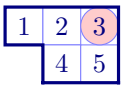

```
\Diagram[show=residues, e=3]{3,2,1}
```

1c. **Skew tableau.**

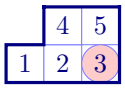


```
\tikzset{R/.style={fill=red!20,circle}}
\SkewTableau{2,1^2}{12[R]3,45,67}
\SkewTableau[french,skewwall]{2,1^2}{12[R]3,45,67}
```

1d. **Shifted tableau.**

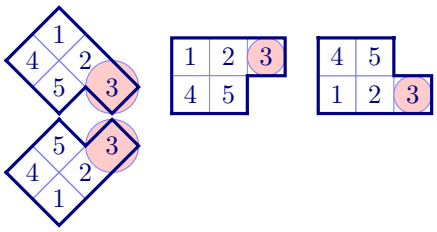


```
\tikzset{R/.style={fill=red!20,circle}}
\ShiftedTableau{12[R]3,45}
```



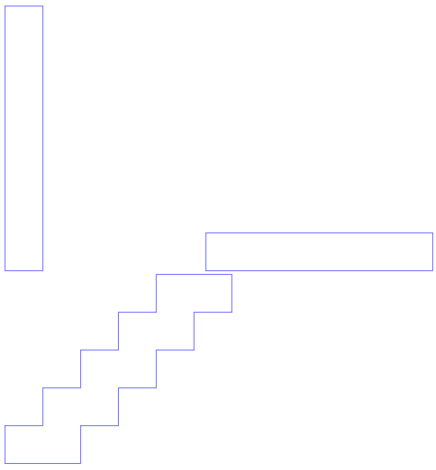
```
\tikzset{R/.style={fill=red!20,circle}}
\ShiftedTableau[french]{12[R]3,45}
```

1e. **Tabloids.** A *tabloid* is an equivalence class of tableau where two tableaux are equivalent if they have the same entries in each row, up to a permutation. In the literature, tabloids are drawn with (outer wall) lines drawn above and below each row, and no side boundary walls. The `\Tabloid` command functions in the same way as the `\Tableau` command, except that it draws tabloids.

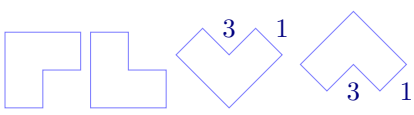


```
\tikzset{R/.style={fill=red!20,circle}}
\Tabloid[australian]{12[R]3,45}
\Tabloid[english]{12[R]3,45}
\Tabloid[french]{12[R]3,45}
\Tabloid[ukrainian]{12[R]3,45}
```

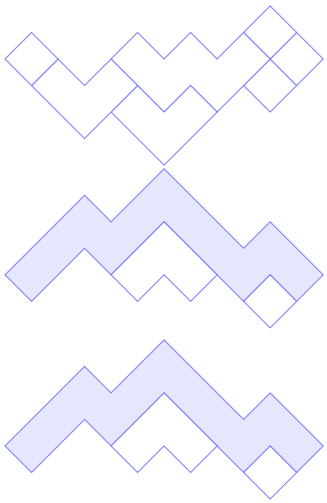
1f. **Ribbon tableaux.** A *ribbon* in a tableau, or in a diagram, is a connected strip of nodes that does not contain a ⊞-square. The *head* of a ribbon R is the node $(r, c) \in R$ such that $r' > r$ and $c' < c$ whenever $(r', c') \in R$ and $(r', c') \neq (r, c)$.



```
\RibbonTableau{11rrrrrr}
\RibbonTableau{16ccccc}
\RibbonTableau{1{10}crcrcrcrc}
```

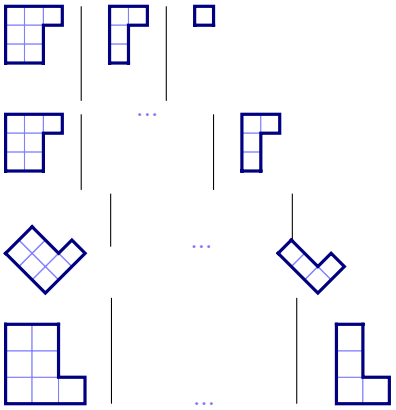


```
\RibbonTableau{
[fill=blue!20|1]12[fill=red!20|2]c[fill=yellow!20|3]r }
\RibbonTableau[french]{ [|1]12[|2]c[|3]r }
\RibbonTableau[ukrainian]{ [|1]12[|2]c[|3]r }
\RibbonTableau[australian]{ [|1]12[|2]c[|3]r }
```

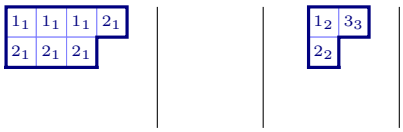


```
\RibbonTableau[skew={4,1^2},ukrainian]
  {15,16,26,25crcr,23cr,42cr,61}
\RibbonTableau[skew={4,1^2},australian]
  {fill=blue!10|16crcccrrcrr,26,34cr}
\RibbonTableau[australian]
  {fill=blue!10|16crcccrrcrr,26,34cr}
```

1g. **Multidiagrams and multitableaux.**



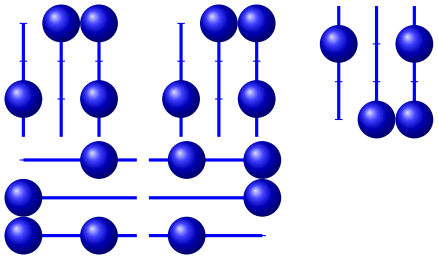
```
\Multidiagram[scale=0.5]{3,2^2
\Multidiagram[scale=0.5,rows=3,
\Multidiagram[ukrainian,scale=
cols=3]{3,2^2||...||2,1^2}
\Multidiagram[french,scale=0.7,
```



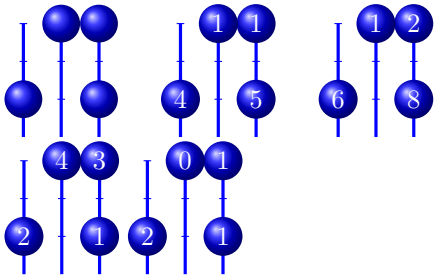
```
\Multitableau[rows=3,scale=0.8,font=\tiny]
    {{1_1}{1_1}{1_1}{2_1},{2_1}{2_1}{2_1} ||
     {1_2}{3_3},{{2_2}} ||
     {1_3}{1_3},{{2_3}}}
```

```
\Multitableau[rows=3,scale=0.8,font=\tiny]
            {123,45,67||89,{10},{11}||...||{12}}
```

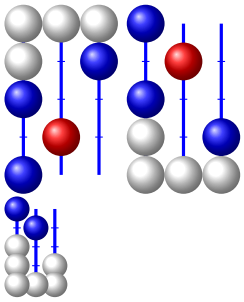
2. Abacuses



```
\Abacus{3}{5,4,1,1}
\Abacus[south]{3}{5,4,1,1}
\Abacus[north]{3}{5,4,1,1}
\Abacus[east]{3}{5,4,1,1}
\Abacus[west]{3}{5,4,1,1}
```



```
\Abacus{3}{5,4,1,1}
\Abacus[show=shape,text=white]{3}{5,4,1,1}
\Abacus[show=beta,text=white]{3}{5,4,1,1}
\Abacus[show=row,text=white]{3}{5,4,1,1}
\Abacus[show=residue,text=white]{3}{5,4,1,1}
```

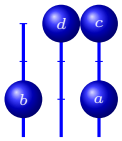


```
\tikzset{R/.style={ball color=red},
         S/.style={ball color=white}}


\Abacus{3}{5,[R]4,1^2,[S]0^4}
\Abacus[north]{3}{5,[R]4,1^2,[S]0^4}

\Abacus[north, scale=0.5, starstyle={ball
color=white}]{3}{5,4,*1^2,*0^4}
```

An abacus:



```
\Abacus[text=white,font=\tiny]{3}{5_a,4_b,1_c,1_d}
```

Done!

3. Dynkin quivers?

4. Braid diagrams and KLRW diagrams?

5. aTableau settings

6. The code

Every command provided by aTableau uses TikZ, with the bulk of the code in the package being written in LaTeX3. Most of the tableau commands are special cases of the `\Tableau` command. Here is a dictionary:

Command	Equivalent `\Tableau` command
`\ShiftedTableau{...}`	`\Tableau[shifted]{...}`
`\SkewTableau{$\mu$}{...}`	`\Tableau[skew=$\mu$]{...}`
`\Tabloid` {...}	`\Tableau[tabloid]{...}`

The `\Diagram` command also invokes the `\Tableau` command, however, this requires some involved internal trickery. For example, `\Diagram{3,2^2}` is the same as `\Tableau{...,..,..}`.

Use the `\EAbacus` command to a typeset an abacus

Index